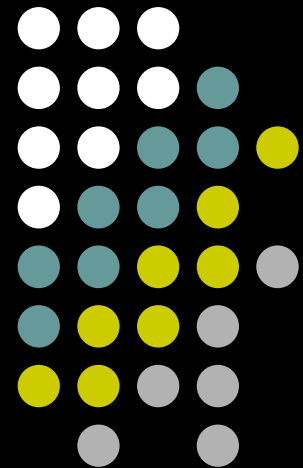


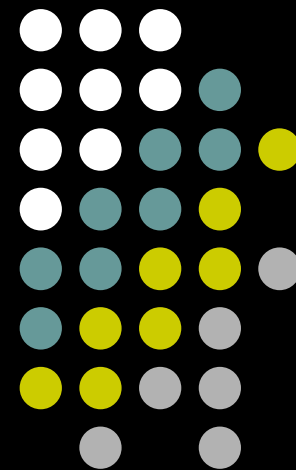
第5章 循环结构程序设计

Part A

- 5.1 为什么需要循环控制
- 5.2 用while语句实现循环
- 5.3 用do...while语句实现循环
- 5.4 用for语句实现循环
- 5.5 循环的嵌套



5.1 为什么需要循环控制



需要重复处理的问题



- 在日常生活中或是在程序所处理的问题中常常遇到需要重复处理的问题
 - 要向计算机输入全班50个学生的成绩
 - 分别统计全班50个学生的平均成绩
 - 求30个整数之和
 - 教师检查30个学生的成绩是否及格

【在没有循环结构（或称重复结构）的情况下】
只能编写若干个相同或相似的语句或程序段

【例】全班有50个学生，统计各学生三门课的平均成绩。



- 求一个学生平均成绩的程序段

```
scanf(" %f, %f, %f",&s1,&s2,&s3);  
aver=(s1+s2+s3)/3;  
printf("aver= %7.2f",aver);
```

要对50个学生进行相同操作 重复50次



【弊端】工作量大、程序冗长、重复、难以阅读和维护！

循环控制用来处理需要重复

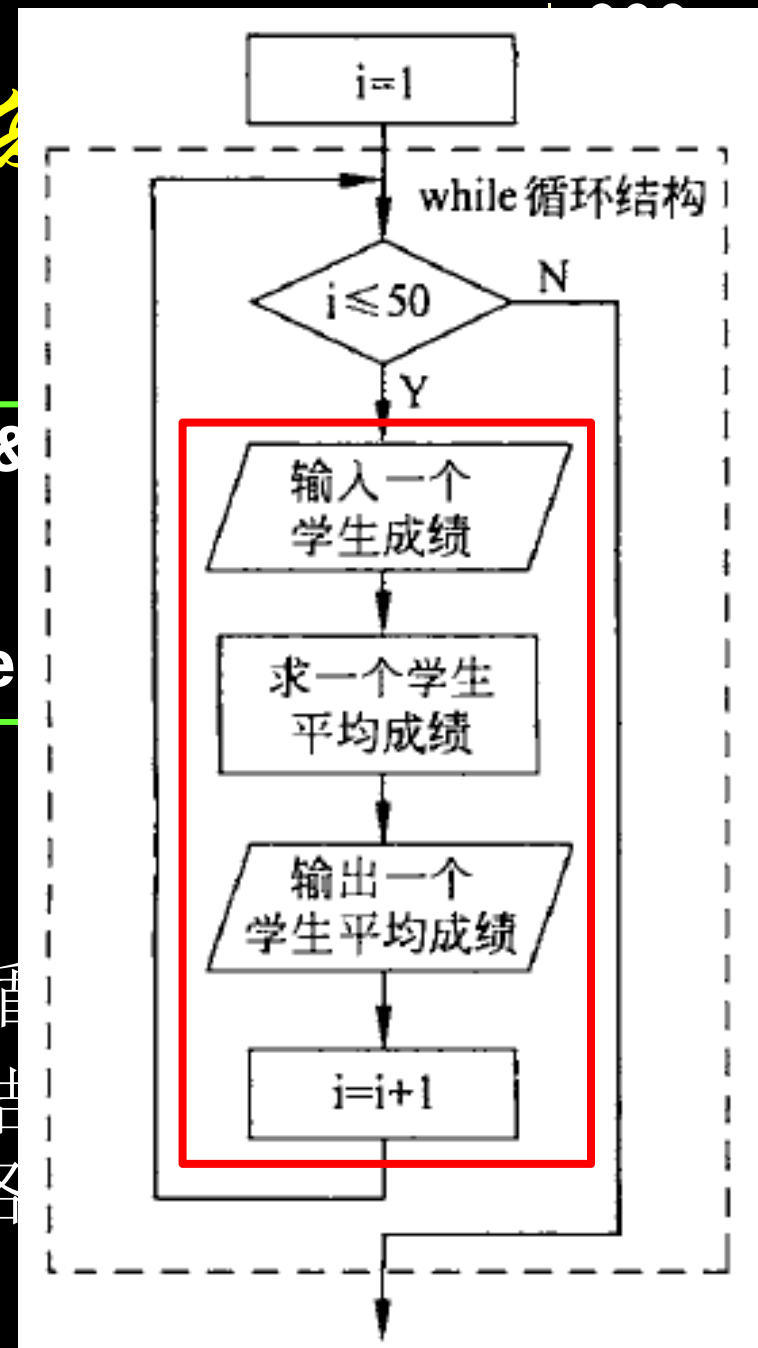
`i=1;` `while` (表达式)

`while (i<=50)` 语句

```
{ scanf("%f,%f,%f",&s1,&s2,&s3);  
  aver=(s1+s2+s3)/3;  
  printf("aver=%7.2f",aver);  
  i++;  
}
```

(流程图见P115 图5.1)

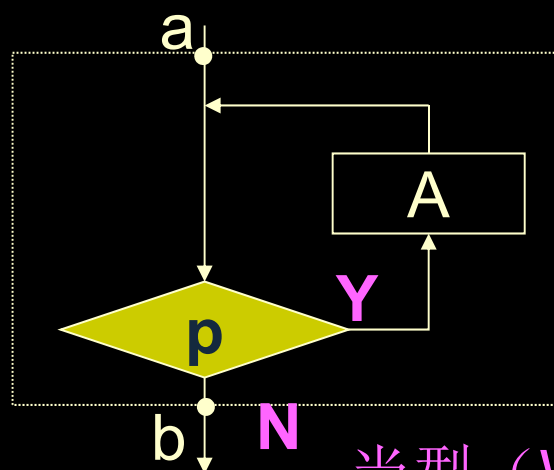
- 大多数的应用程序都会包含循环
- 循环结构和顺序结构、选择结构是程序设计的三种基本结构，它们是各构造单元



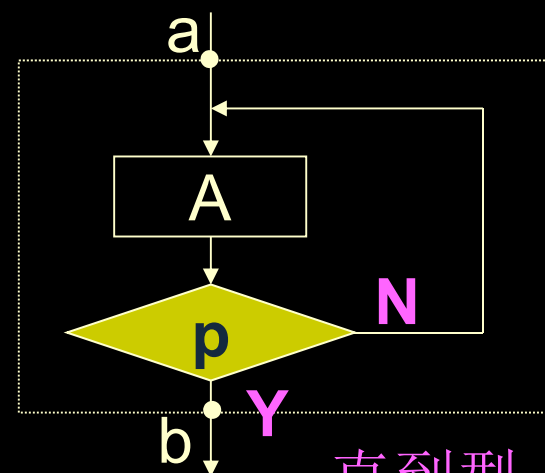
概述



- **循环/迭代**：在满足特定条件时，重复执行一些操作；



当型 (While型)

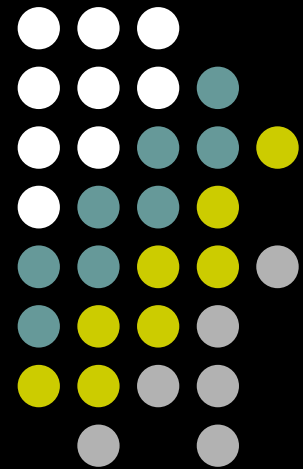


直到型 (Until型)

- 实现循环控制结构的几种方法：

- 1) 用while语句
- 2) 用do-while语句
- 3) 用for语句
- 4) 用goto语句和if语句构成循环*

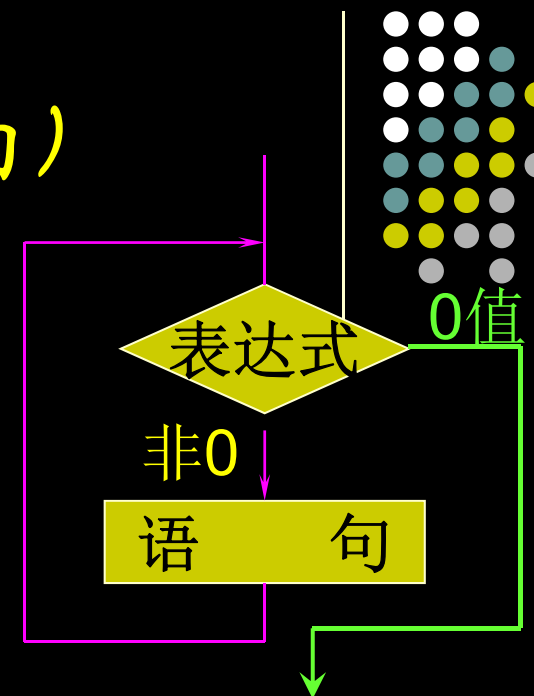
5.2 用while语句实现循环



while语句（“当型”循环结构）

while语句的一般形式如下：

while（**表达式**） 循环条件
 语句 循环体 表达式



【功能】

步骤1：计算表达式的值；

步骤2：如果表达式的值为**真**（即**非0**），则执行“语句”，
然后**回到步骤1**；

否则（即值为**假**，即**0**），不执行“语句”，
并**结束循环**。

while循环的特点（与**do...while**对比）：

先判断条件表达式，后执行循环体语句

【注意】循环体可能被执行多次，也可能一次也不被执行！⁸

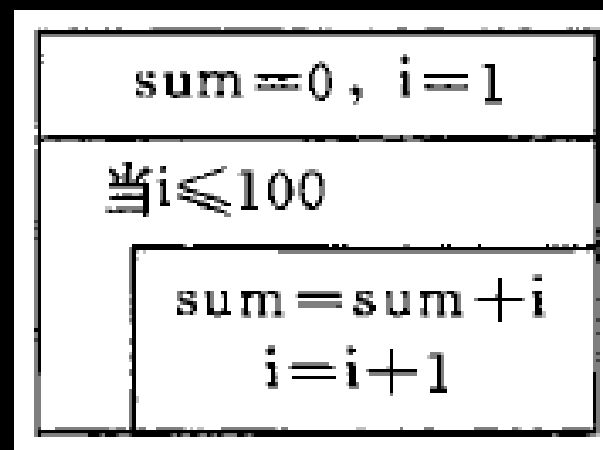
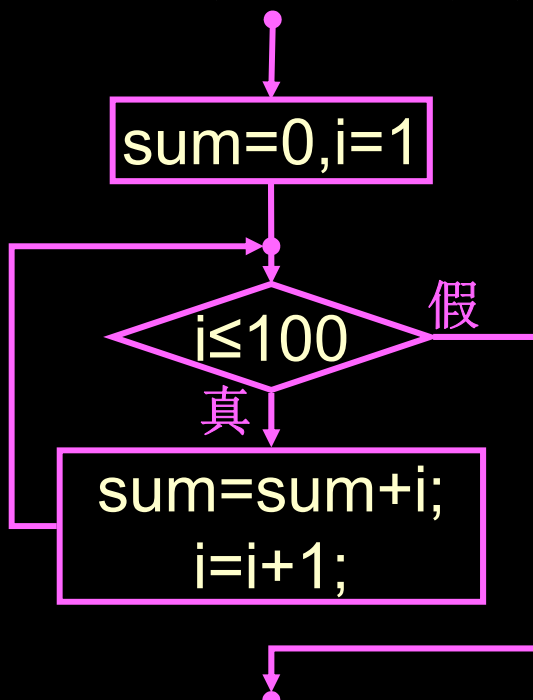
【例5.1】求 $1+2+3+\dots+100$ ，即

$$\sum_{n=1}^{100} n$$



【解题思路】

- 这是累加问题，需要先后将100个数相加
- 要重复100次加法运算，可用循环实现
- 后一个数是前一个数加1而得
- 加完上一个数*i*后，使*i*加1可得到下一个数



```
#include <stdio.h>
```

```
int main()
```

```
{
```

赋初值不能少!

```
    int i=1, sum=0;
```

```
    while (i<=100) ← 进入循环体前先检查条件是否成立  
                        (表达式是否不为0)
```

```
    {    sum=sum+i;
```

```
        i++;
```

```
    }
```

复合语句

循环体结束之前，检查条件是否依然成立，如果是，再次执行循环体

```
    printf("sum=%d\n", sum);
```

```
    return 0;
```

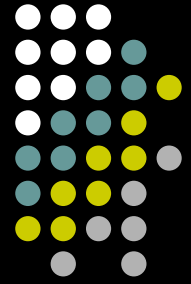
```
}
```

```
sum=0, i=1
```

```
当i≤100
```

```
sum=sum+i
```

```
i=i+1
```



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int i=1, sum=0;
```

```
    while (i<=100)
```

```
    {    sum=sum+i;
```

```
        i++;    不能丢，否则循环永不结束
```

```
    }
```

```
    printf("sum=%d\n", sum);
```

```
    return 0;
```

```
}
```

|sum=5050

循环体中应有能使循环趋向于结束的语句！



计数循环



- 计数循环，循环执行次数是能预先确定的

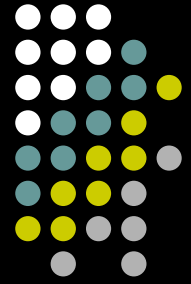
```
int end = 100;  
int count = 1; //初始化  
while (count <= end) //循环条件  
{  
    printf("%d\t", count);  
    count++; //更新计数  
}
```

计数循环的三个基本步骤:

- ① 循环之前，必须初始化一个计数器；
- ② 循环判断条件通常是计数器与某个有限值的比较
- ③ 每次执行循环，（在循环体或表达式中）计数器的值都要更新（朝终止方向）

【注意】对循环所涉及的变量的初始值、循环的终止条件（是否包含等号）等，应仔细推敲。

- 不确定循环，循环体执行的次数不能预先确定的



例：文件复制 (K & R, P10)

- 功能：把输入复制到输出（一次一个字符）

- 版本1:

```
#include <stdio.h>
int main( )
{ int c;
  c = getchar( );
  while (c != EOF) {
    putchar(c);
    c=getchar( );
  }
  return 0;
} 【代码见c5_z2.c】
```

读入一个字符

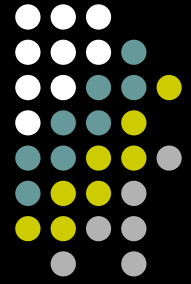
当该字符不是文件结束符时

输出该字符

读入下一个字符

- 版本2:

```
#include <stdio.h>
int main( )
{ int c;
  while ((c = getchar( )) != EOF)
    putchar(c);
  return 0;
} 【代码见c5_z3.c】
```



例：字符计数 (K & R, P12)

- 功能：统计输入的字符数

```
#include <stdio.h>
```

```
int main( )
```

```
{ long nc;
```

```
    nc = 0;    while (getchar( ) != '\n')
```

```
    while (getchar( ) != EOF)
```

```
        ++nc;
```

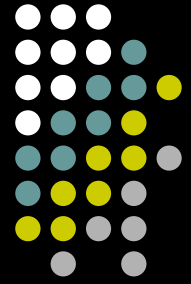
```
    printf("%ld\n",nc);
```

```
    return 0;
```

```
}
```

【思考】若只统计一行输入的字符数，该如何修改？

【代码见c5_z4.c】



例：行计数 (K&R, P13)

- 功能：计算输入中的行数（每行均以换行符结束）

```
#include <stdio.h>
```

```
int main( )
```

```
{ int c, nl;
```

```
  nl = 0;
```

```
  while ((c = getchar( )) != EOF)
```

```
    if (c == '\n' )
```

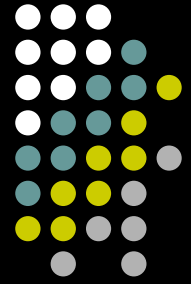
```
        ++nl;
```

```
  printf( "%d\n" , nl );
```

```
  return 0;
```

```
}
```

【代码见c5_z5.c】



while 语句小结

While语句的使用中的注意事项:

★ 循环体中只能出现一个语句，若有多个操作，必须采用复合语句。

如:

```
while (i<=100)
    sum=sum+i;
    i++;
```

又如:

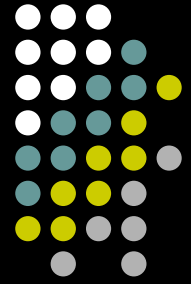
```
while (<=100);
{ sum=sum+i;
  i++; }
```

★ 在循环体中一定要有能使循环趋于终止的语句，否则会出现死循环。

如:

```
i=10;
while (i<=10) printf("%i\n", i);
```


轻松一刻——循环



- A programmer's wife asks him to go to the store and pick up a stick of butter, and while he's there, pick up eggs. He never returned.

① go to the store

② pick up a stick of butter

③ while he's there

pick up eggs



代码人生



```
int memory=0;  
int happiness=0;  
int sadness=0;
```

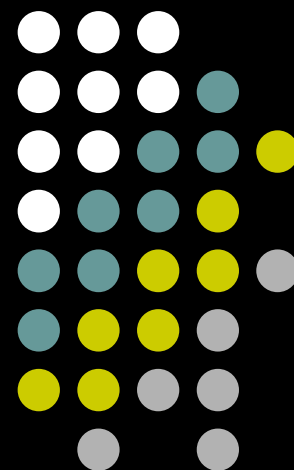
```
while(alive)  
{  
    memory++;  
    happiness++;  
    sadness++;  
}
```

```
memory=0;  
happiness=0;  
sadness=0;  
return null;
```

死去元知万事空，但悲不定alive

alive并没有初始化～
直接胎死腹中.....好悲伤

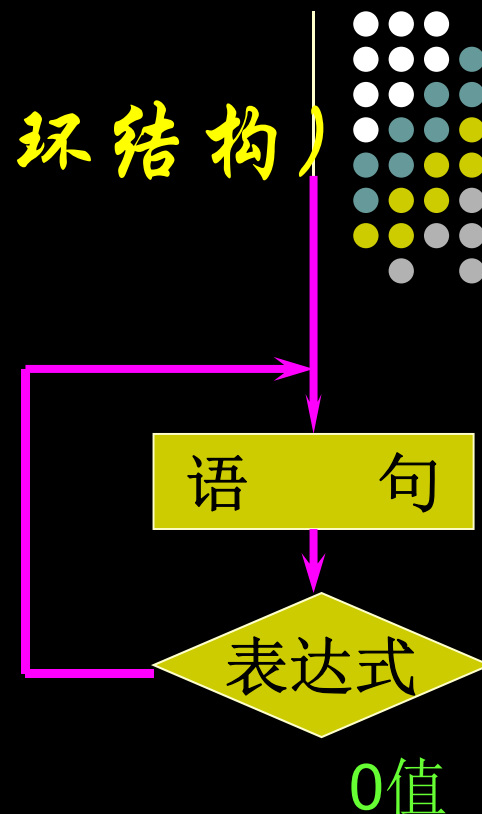
5.3 用 do...while 语句 实现循环



do...while语句 (“直到型”循环结构)

while语句的一般形式如下：

do 循环体
 语句 循环条件表达式
while (表达式)



【功能】

步骤1：执行“语句”

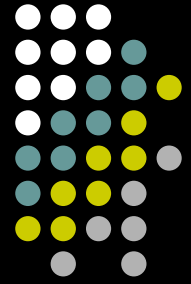
步骤2：计算表达式的值；

步骤3：如果表达式的值为真（即非0），回到步骤1；
否则（即值为假，即0），结束循环。

do...while循环的特点（与**while**对比）：

先无条件执行循环体一次，再判断条件表达式是否成立

【注意】无论如何，循环体至少会被执行一次！



【例】输出1~100

```
int i=1;
do
{
    printf("%d",i++);
}
while(i<=100);
```

```
do
    printf("%d",i++);
while(i<=100);
```

- 用花括号把循环体括起来，程序更清晰、易读。

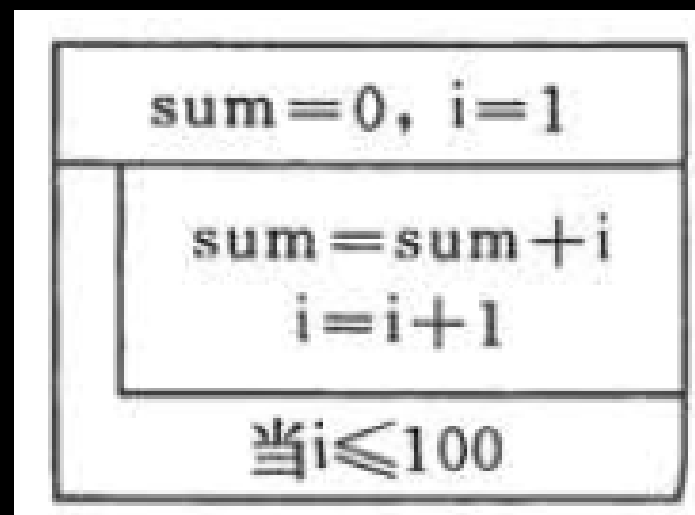
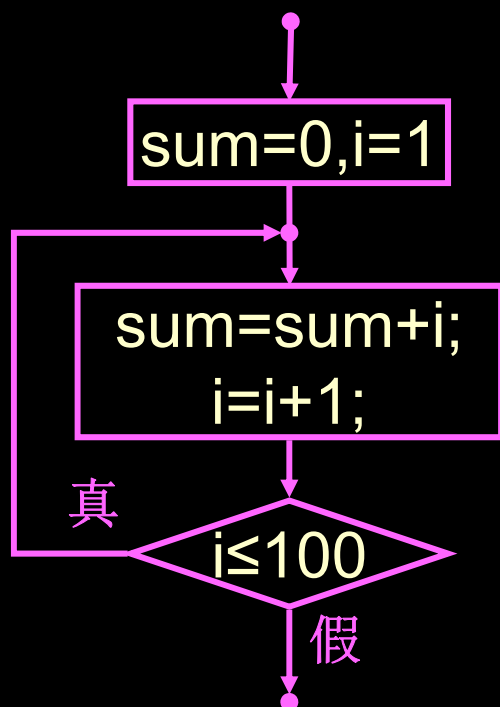
【例5.2】用do...while语句求
1+2+3+...+100, 即

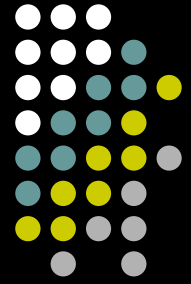
$$\sum_{n=1}^{100} n$$



【解题思路】

- 与例5.1类似, 用do...while循环实现

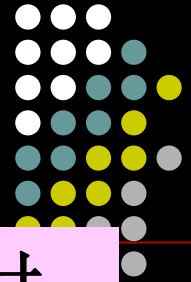




```
#include <stdio.h>
int main()
{ int i=1,sum=0;
  do
  {
    sum=sum+i;
    i++;
  }while ( i<=100 );
  printf("sum=%d\n",sum);
  return 0;
}
```

sum=5050

【例5.3】while和do...while循环的比较



当while后面的表达式一开始的值为“真”时，
两种循环得到的结果相同；否则不相同

```
scanf("%d",&i);  
while(i<=10)  
{  
    sum=sum+i;  
    i++;  
}  
printf("sum=%d\n",sum);
```

```
i=?1  
sum=55
```

```
i=?11  
sum=0
```

```
scanf("%d",&i);  
do  
{  
    sum=sum+i;  
    i++;  
}while(i<=10);  
printf("sum=%d\n",sum);
```

```
i=?1  
sum=55
```

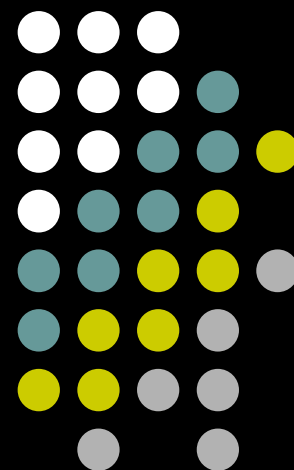
```
i=?11  
sum=11
```


do-while 语句小结



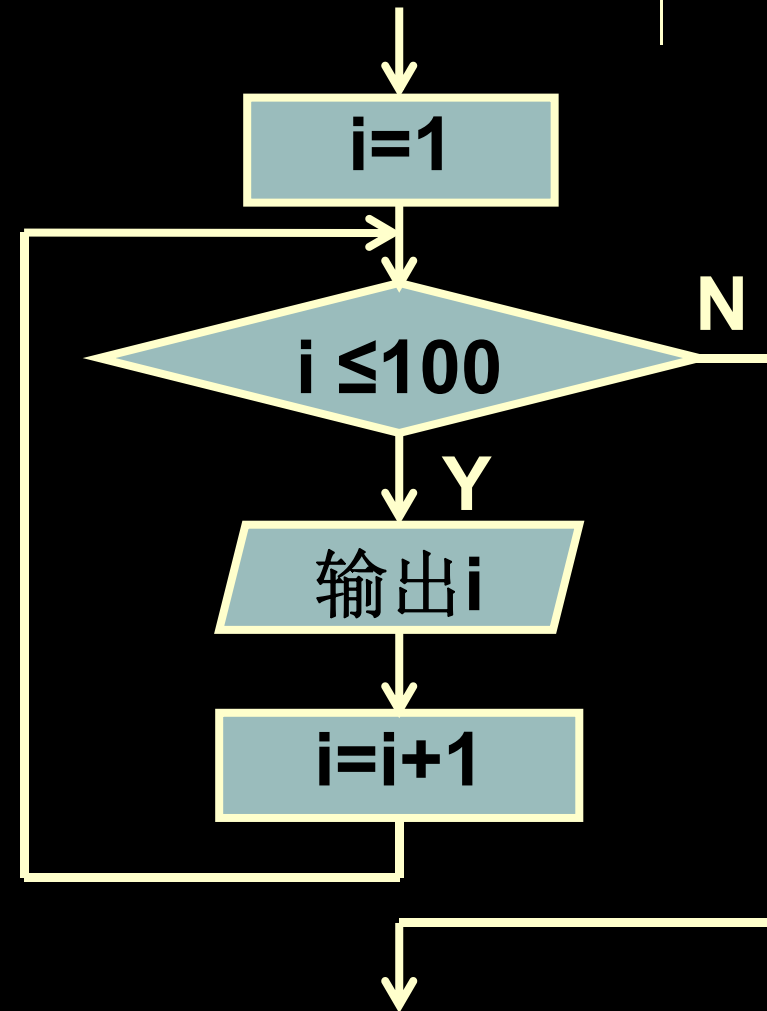
- do-while语句和while语句的区别：
 - 1) 执行循环体和判断表达式的先后顺序。
 - 2) do-while至少执行循环体一次；而while语句可能一次也不执行循环体。
- do-while语句结构可以转换成while结构，两者的控制程序执行流程的能力完全等价。

5.4 用for语句实现循环



【例】输出1~100

```
for (i=1;i<=100;i++)  
{  
    printf("%d ", i );  
}
```



for 语句

其一般结构是：

for (表达式1;表达式2;表达式3)

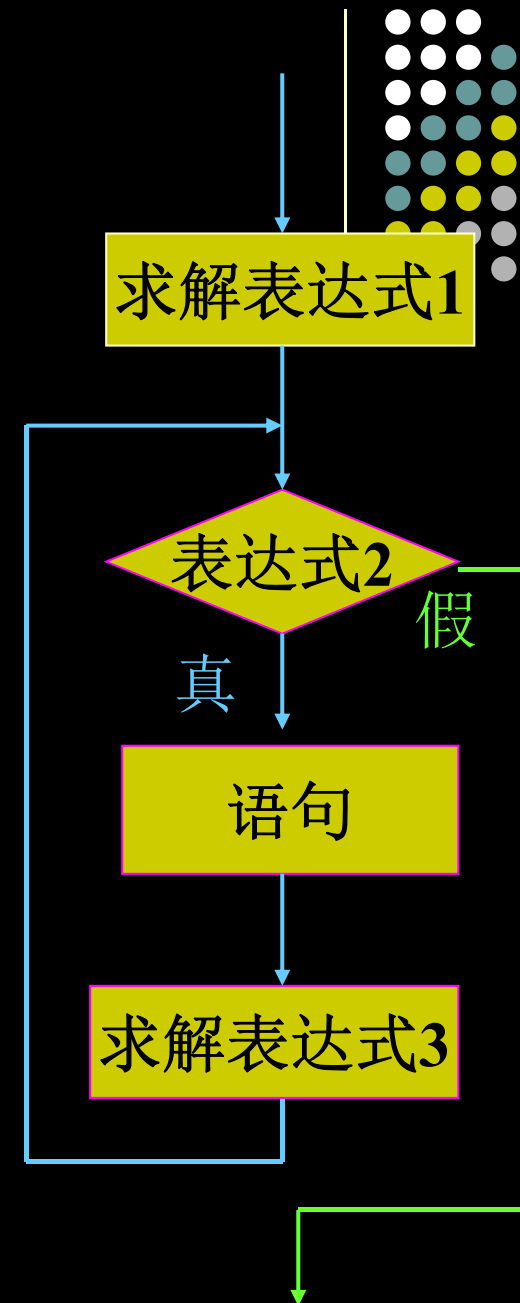
循环体

语句

设置初始条件，
只在最开始执行一次。可以为零个、一个或多个变量设置初值（逗号隔开）

循环条件表达式，用来判定是否继续循环。在每次执行循环体前先对此表达式求值，决定是否继续执行循环

作为循环的调整器，例如使循环变量增值，它是在每次执行完循环体后才求值的



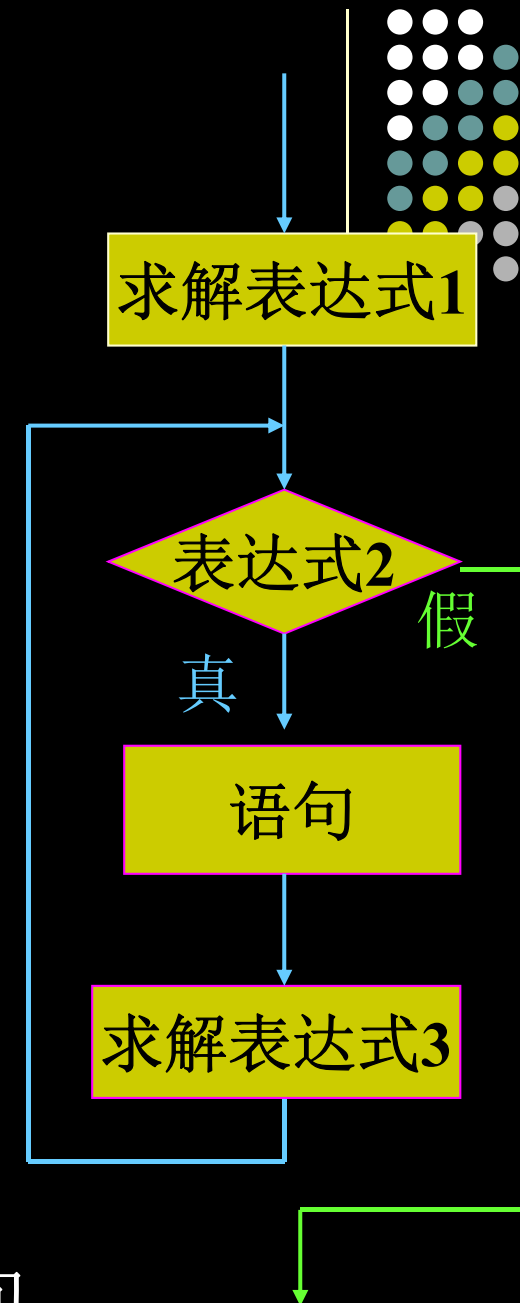
for 语句（续）

其一般结构是：

for (表达式1;表达式2;表达式3)
语句

➤ **for**语句的执行过程：

- (1) 先求解表达式1
- (2) 求解表达式2，若其值为真，执行“语句”，然后执行第(3)步；若为假，则结束循环，转到第(5)步
- (3) 求解表达式3
- (4) 转回步骤(2)，继续执行
- (5) 循环结束，执行**for**语句下面的一个语句



for 语句（续2）

其一般结构是：

for (表达式1;表达式2;表达式3)

语句 逗号表达式

【例】**for (i=1,sum=0; i<=100; i++)**

sum=sum+i;

$$\sum_{i=1}^{100} n$$

- 等价的while循环形式：

表达式1;

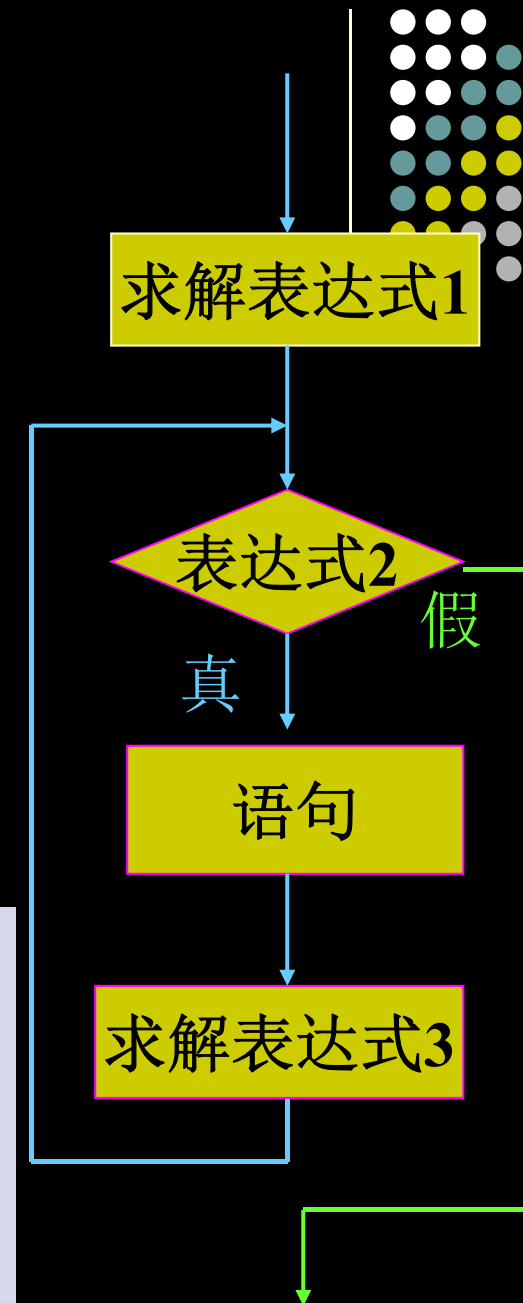
while (表达式2)

{ 语句;

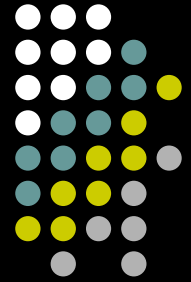
表达式3;

}

```
i=1,sum=0;
while (i<=100) {
    sum=sum+i;
    i++;
}
```



for 语句的使用（各种变形）



`sum=0; for (i=1; i<=100; i++) sum=sum+i;`

- **表达式1省略**：通常在for语句之前对循环变量赋初值。

如：`sum=0; i=1; for (; i<=100; i++) sum=sum+i;`

- **表达式2省略**：不判断循环条件，即表达式2永真，循环无终止（除非循环体中设法跳出，如用break、goto、return等）。

如：`sum=0; for ( i=1; ;i++) {sum=sum+i; if (i==100) break; }`

- **表达式3省略**：通常在循环体内设法保证循环能正常结束。

如：`sum=0; for ( i=1; i<=100; ) sum=sum+i++;`

- **表达式1和表达式3省略**：完全等同于while语句。

如：`i=1; for (; i<=100; ) sum=sum+i++;`

- **三个表达式全省略**：类似表达式2省略。

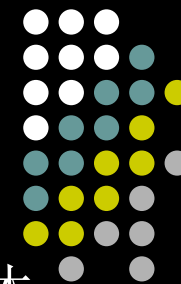
如：`i=1; for ( ;; ) { sum=sum+i++; if (i>100) break; }`

注意：括号内的分号不能省略！

- 其他变形：如使用逗号表达式、无循环体等等。

如：`for ( i=1, sum=0; i<=100; sum+=i, i++ );`

如：`for ( i=100, sum=0; i>=1; sum+=i, i-- );`



for 语句的使用（各种变形）

- 表达式一般是关系表达式或逻辑表达式，但也可以是数值表达式或字符表达式，只要其值为非0，就执行循环体。

【例c5_z7.c】输入字符并将它们的ASCII码相加，直到输入一个“换行”符为止。

```
#include <stdio.h>
```

```
int main( )
```

```
{ char c; int i;
```

```
    for ( i=0; ( c = getchar() ) != '\n'; i+=c );
```

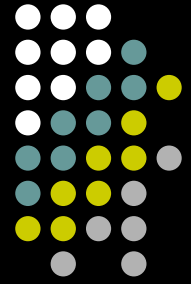
```
    printf( "%d", i );
```

```
    return 0;
```

```
}
```

- C99允许在for语句的“表达式1”中定义变量并赋初值

```
    for (int i=1; i<=100; i++) sum=sum+i;
```

例：字符计数 (K & R, P12)

- 功能：统计输入的字符数

- 版本1：

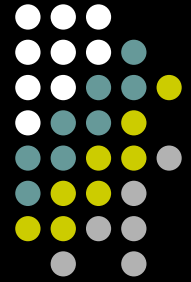
```
#include <stdio.h>
int main( )
{ long nc;
  nc = 0;
  while (getchar( ) != EOF)
    ++nc;
  printf("%ld\n",nc);
  return 0;
}
```

【代码见c5_z4.c】

- 版本2：

```
#include <stdio.h>
int main( )
{ long nc;
  for (nc = 0; getchar( ) != EOF;
        ++nc);
  printf("%ld\n",nc);
  return 0;
}
```

【代码见c5_z8.c】



for 语句的使用（各种变形）*

- for语句中浮点型数据的使用

【例c5_z9.c】输出从1.0~2.0，间隔0.1的各个数。

```
#include <stdio.h>
```

```
int main()
```

```
{ double x;
```

这个程序在有些机器上运行可能不会打印出**2.0!**

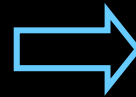
```
for (x=1.0; x<=2.0; x+=.1) {
```

```
    printf("%.1f\n",x);
```

```
}
```

```
return 0;
```

```
}
```



```
for (x=10; x<=20; x+=1) {
```

```
    printf("%.1f\n",x/10.0);
```

- 尽管没有任何明显的理由要求for循环的控制变量必须是整型，但由于浮点型并不是百分之百精确的，为了避免类似问题，**循环控制变量最好用整型**。

- 除了for循环，其它一些情况下同样需要警惕“浮点型数据比较”可能引发的问题。

for 语句的使用（各种变形）



- for 语句与其他语言中的for有不同的地方
 - 三个表达式都可以是任意的表达式
 - 循环变量和循环条件可在循环体内修改
 - 循环变量的值在循环结束时仍保持着
- 尽管C语言的for语句功能比其它语言要强，使用上也更灵活，但若是过分地利用它的这些特性会使for语句显得杂乱，可读性降低。
 - 切莫死记硬背，只须领会其精神实质，学会灵活使用！

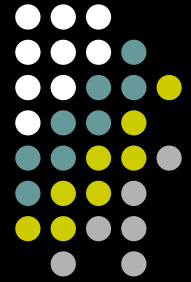


for语句小结

- for语句不仅可以用于循环次数已经确定的情况，还可以用于循环次数不确定而只给出循环结束条件的情况
- for语句完全可以代替while语句

不可乱用分号——女生节

```
for(int i=0;i<forever;i++);  
    printf("I love my girl");
```



一个分号说明的悲伤故事....



- 男孩给喜欢很久的女孩写了一封情书

```
for(int day = SOMEDAY; day < FOREVER; day++)  
{  
    love++;  
}
```

意思是：情不知所起，一往情深.....

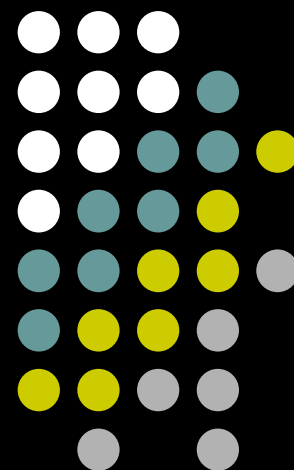
- 女孩的回答是 在for那行后面加了一个分号

```
for(int day = SOMEDAY; day < FOREVER; day++);  
{  
    love++;  
}
```

“分”了，自然就不会再爱了。

虽然离love++就差一步，可惜进入了死循环。。。这是一个悲哀的故事。

5.5 循环的嵌套



循环的嵌套



循环的嵌套：一个循环体中又包含另一个完整的循环结构。

- “外层循环”与“内层循环”
- 三种循环可以互相嵌套（见P121）
- 内嵌的循环结构一定要完整的！不允许出现循环体交叉现象。

【例c5_z6.c】输出九九乘法表

```
#include <stdio.h>
```

```
int main()
```

```
{ int i=1, j;
```

```
  while (i<=9) {
```

```
    j=0;
```

i

```
    while ( j++ < 9 )
```

```
        printf("%1d*%1d=%2d  ", i, j, i*j);
```

```
    i++;
```

```
    printf("\n");
```

```
}
```

```
  return 0;
```

```
}
```

【思考】如何修改使其只输出矩阵的上三角或下三角部分？

合法嵌套形式举例



(1) while()

```
{  
  :  
  while()  
  {...}  
}
```

} 内层循环

(2) do

```
{  
  :  
  do  
  {...}  
  while()  
}
```

} 内层循环

} while()

(3) for(;;)

```
{  
  :  
  for(;;)  
  {...}  
}
```

} 内层循环

(4) while()

```
{  
  :  
  do  
  {...}  
  while();  
  :  
}
```

} 内层循环

(5) for(;;)

```
{  
  :  
  while()  
  {...}  
  :  
}
```

} 内层循环

(6) do

```
{  
  :  
  :  
  for(;;)  
  {...}  
}
```

} 内层循环

} while();

逗号运算符和逗号表达式



- **逗号运算符**：又称为“**顺序求值运算符**”，用来将两个表达式连接起来，
 - 如： $3+5$ ， $6+8$
- **逗号表达式**的一般形式：
表达式1， 表达式2， $\dots$ ， 表达式n
- 逗号表达式的求解过程：依次对表达式1、表达式2、 $\dots$ 、表达式n求值，并将最后一个表达式n的值作为整个逗号表达式的值。
 - 如： $a=3*5$ ， $a*4$
- 逗号运算符的**优先级**是所有运算符中**最低**的，且是从左到右结合的。

逗号运算符和逗号表达式 (2)



- 逗号表达式最常用于循环语句(for语句)中

【注意】并不是任何地方出现的逗号都是作为逗号运算符！

- 区分：

```
printf("%d, %d, %d", a, b, c);  
printf("%d, %d, %d", (a, b, c), b, c);
```

返回